

Robothons- Web

Abhishek Suresh(as18757)
Sai Krishna Bommavaram(sb8944)
Parth Chaturvedi(pc3457)

Supervised by:
Prof Jean-Claude Franchitti
Joanna Gilberti

Introduction

1. About
2. Business Use Case
3. Execution Workflow
4. CITI Competition Platform
5. C++ Bot Architecture
6. Java Bot Architecture
7. RabbitMQ C Advantages
8. Other Language Bot Architecture
9. Future Work
10. Conclusion



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES



About Robothon



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES

Introduction to Robothon

Robothons is a multi-industry competition and simulation support platform that enables the development of trading bots and health bots through crowdsourcing and machine learning.

Robothons - Global Markets (CITI)

This track of Robothons focuses on crowdsourcing the development of trading bots using historical stock market data. The bots are tagged by market regime and catalogued via the MetaML platform, and participants complete tasks such as generating orders and trading on a dynamic stock market simulation platform.

Robothons - Healthcare (MEYERS)

This track of Robothons focuses on crowdsourcing the development of health bots using real-time data from the AppEase watch application. The health bots are tagged by stress regime and catalogued via the MetaML platform, and they evolve through dynamic stress simulation platforms to be used on devices like the AppEase watch or AR devices.



Business Use Case



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES

Challenge

Financial markets demand rapid response to market events. Strategies need to be updated frequently to account for new data.

Solution

Use a hybrid system where C++ handles high-speed trading operations and Python allows dynamic strategy customization.

Objectives

Enhance trading efficiency, improve profit margins, and reduce time-to-market for new strategies.

C++ Component

Responsible for high-speed trading operations, ensuring fast execution of trades and order management.

Python Component

Enables dynamic customization of trading strategies, allowing for agile adaptation to market changes.

Hybrid Approach

Combines the strengths of C++ (performance) and Python (flexibility) to create a powerful trading system.



Execution Workflow



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES

Bot Initialization

Define trading parameters such as the trading strategy, starting capital, and other configurations to set up the bot.

Load Market Events

Connect to various data feeds, such as stock quotes, news, and economic indicators, and queue the incoming market events for processing.

Strategy Execution

The bot's management class analyzes the incoming data and computes the optimal trading strategy based on the defined parameters.

Trade Execution

The bot executes trades based on the recommendations from the strategy execution step and records the results of the trades.



CITI Competition Platform



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES

Participant Bot Development

Participants develop and submit automated trading bots to compete in the CITI competition platform.

Bot Validation

The submitted trading bots are validated to ensure they meet the competition's requirements and qualify for the competition.

Dynamic Market Simulation

During the competition, the qualified trading bots generate orders and execute trades on a dynamic stock market simulation platform.

Performance Evaluation

The performance of the trading bots is calculated based on predefined metrics, and the winning bot is determined.



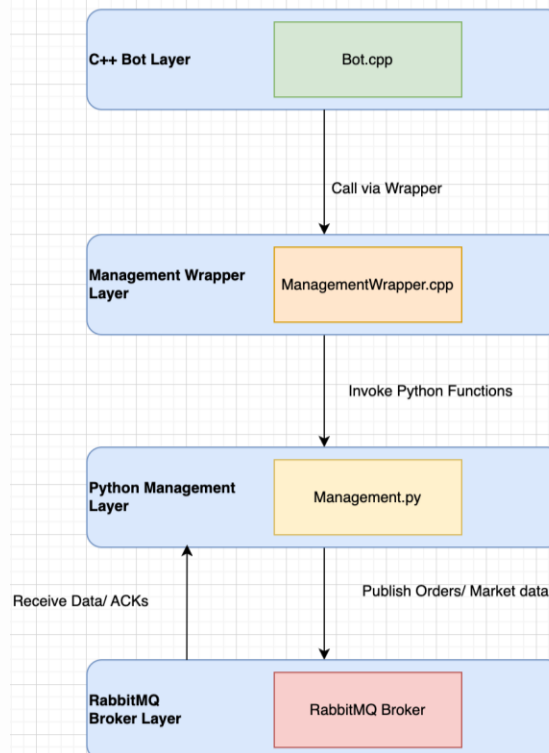
C++ Bot Architecture



NYU

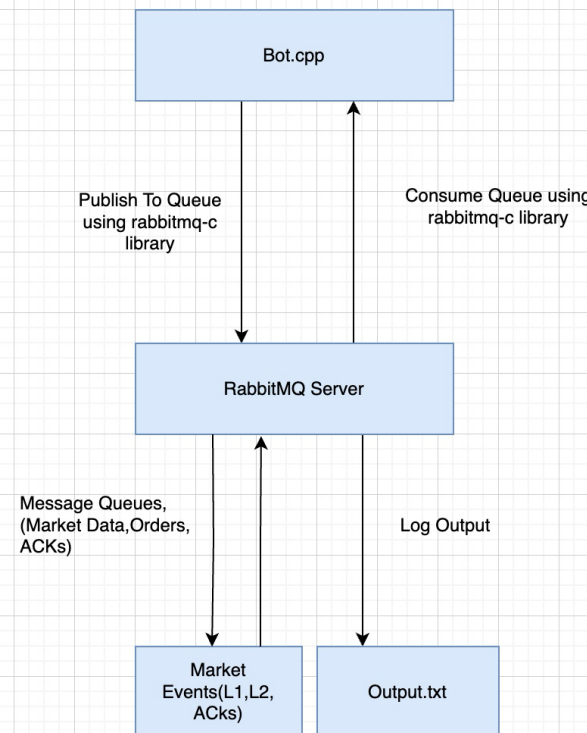
COURANT INSTITUTE OF
MATHEMATICAL SCIENCES

C++ Bot Current Architecture



Initial Architecture

(C++ - Rabbitmq) Architecture



Proposed Architecture

```

(py310) sb8944@robothon:~/RobothonGLBLMkts/ArchSimTrading/Bots/CPP$ ls
ManagementWrapper.cpp ManagementWrapper.h README.md bot bot.cpp check_queue.py
publisher publisher.cpp runme.sh test.cpp
(py310) sb8944@robothon:~/RobothonGLBLMkts/ArchSimTrading/Bots/CPP$ sh runme.sh
RABBITMQ_CFLAGS: -I/robothon/sb8944/rabbitmq-c/include -I/robothon/sb8944/rabbitmq-c/
include/rabbitmq-c -I/robothon/sb8944/rabbitmq-c/build/include/rabbitmq-c
RABBITMQ_LIBS: -L/robothon/sb8944/rabbitmq-c/build/librabbitmq -lrabbitmq
Cleaning up old executable...
Compiling bot.cpp...
In file included from bot.cpp:1149:0:
/robothon/sb8944/rabbitmq-c/include/amqp.h:10:5: warning: #warning "amqp.h is depreca
ted, use rabbitmq-c/amqp.h instead." [-Wcpp]
# warning "amqp.h is deprecated, use rabbitmq-c/amqp.h instead."
~~~~~
In file included from bot.cpp:1150:0:
/robothon/sb8944/rabbitmq-c/include/amqp_tcp_socket.h:10:5: warning: #warning "amqp_t
cp_socket.h is deprecated, use rabbitmq-c/tcp_socket.h instead." [-Wcpp]
# warning "amqp_tcp_socket.h is deprecated, use rabbitmq-c/tcp_socket.h instead."
~~~~~
/usr/bin/ld: warning: /anaconda/envs/py310/lib/libstdc++.so: unsupported GNU_PROPERTY
_TYPE (5) type: 0xc0010001
/usr/bin/ld: warning: /anaconda/envs/py310/lib/libstdc++.so: unsupported GNU_PROPERTY
_TYPE (5) type: 0xc0010002
/usr/bin/ld: warning: /anaconda/envs/py310/lib/libgcc_s.so.1: unsupported GNU_PROPERT
Y_TYPE (5) type: 0xc0010001
/usr/bin/ld: warning: /anaconda/envs/py310/lib/libgcc_s.so.1: unsupported GNU_PROPERT
Y_TYPE (5) type: 0xc0010002
/usr/bin/ld: warning: /anaconda/envs/py310/lib/libgcc_s.so.1: unsupported GNU_PROPERT
Y_TYPE (5) type: 0xc0010001
/usr/bin/ld: warning: /anaconda/envs/py310/lib/libgcc_s.so.1: unsupported GNU_PROPERT
Y_TYPE (5) type: 0xc0010002
Compilation successful.
Running bot with test parameters...
Parsed Arguments:
Action: buy
Symbol: ZFH0:MB0
Size: 100
Max Time: 60
Strategy: TWAP
Bot ID: JGIl0223_eventID_3
Username: quest
Password: quest
Strategy: TWAP
Starting Money: 1e+09
Penalty: 0.125
Net Worth: 1e+09
Subscribed queue amq.gen-dZkiw4zDKs0HVxghucMC2Q to exchange L2 with key ZFH0:MB0
Subscribed queue amq.gen-SKNOFRj3RqvQ59WYXQmzAQ to exchange Clearing with key JGIl022
3_eventID_3
Updated inventory for ZFH0:MB0: 16
    
```

Execution Result



RabbitMQ C Advantages



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES



C++ Programming Language

C++ is a powerful, general-purpose programming language that is widely used in software development, system programming, and game development.



RabbitMQ Message Broker

RabbitMQ is an open-source message broker that facilitates reliable, efficient, and scalable communication between applications.



Architecture Design

The architectural design of a system is crucial for its scalability, performance, and maintainability, and often involves the integration of various technologies.

As we can see the integration of C++, RabbitMQ, and C in a comprehensive architecture design, highlighting the strengths and applications of these technologies.

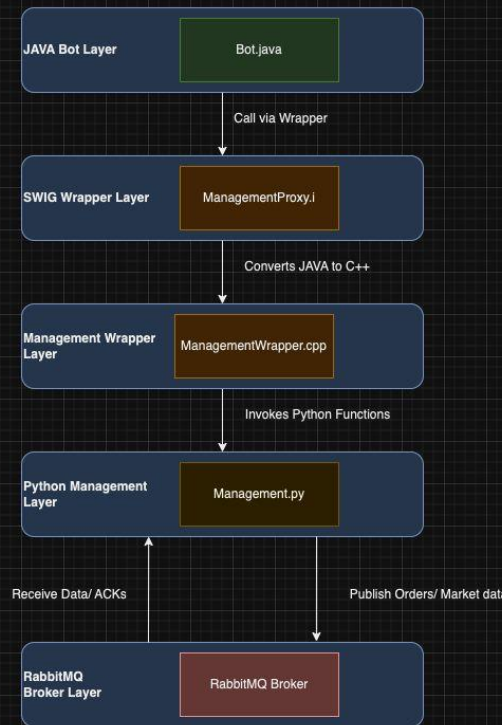
Java Bot Architecture



NYU

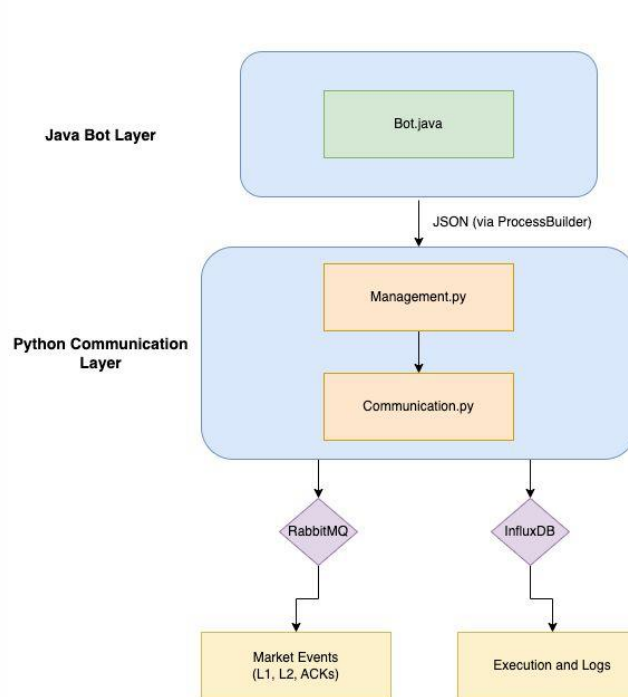
COURANT INSTITUTE OF
MATHEMATICAL SCIENCES

JAVA Bot Current Architecture



Initial Architecture

JAVA Bot Updated Architecture



Proposed Architecture

```

(py310) pc3457@Robothon:~/RobothonGlbMkts/ArchSintTrading/Bots/JAVA$ ./runme.sh
Compiling Java files...
[parsing started RegularFileObject[Bot.java]]
[parsing completed 23ms]
[search path for source files: ./lib/json-20230618.jar]
[search path for class files: /usr/lib/jvm/java-8-openjdk-amd64/re/lib/resources.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/rt.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/sunrt.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/sunspass.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/jsse.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/jce.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/charsets.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/jfr.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/classes.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/sunec_provider.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/zipfs.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/ieee754.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/dnsns.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/jaccess.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/java-atk-wrap.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/sunec.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/locatedata.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/sunec.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/nashorn.jar,/usr/lib/jvm/java-8-openjdk-amd64/re/lib/ext/cldrdata.jar,./lib/json-20230618.jar]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/BufferedReader.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/InputStreamReader.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/OutputStream.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/util/UUID.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Object.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/String.class)]]
[checking Bot]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/Serializable.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/AutoCloseable.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Byte.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Character.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Short.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Long.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Float.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Integer.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Double.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Boolean.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Void.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/System.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/PrintStream.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/FilterOutputStream.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/io/Flushable.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Comparable.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/CharSequence.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Number.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/util/Map.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/util/Collection.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/ProcessBuilder.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/StringBuilder.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/Readable.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/AbstractStringBuilder.class)]]
[loading ZipFileIndexFileObject[/usr/lib/jvm/java-8-openjdk-amd64/lib/ct.sym(META-INF/sym/rt.jar/java/lang/StringBuffer.class)]]
Python script exited with code: 0
Raw response from Python: {"status": "success", "message": "Management.py executed successfully"}
Response from Management.py: {
  "message": "Management.py executed successfully",
  "status": "success"
}
Bot successfully registered in InfluxDB!
Python script exited with code: 0
Raw response from Python: {"status": "success", "message": "Management.py executed successfully"}
Response from Management.py: {
  "message": "Management.py executed successfully",
  "status": "success"
}
Bot successfully registered in InfluxDB!
Python script exited with code: 0
Raw response from Python: {"status": "success", "message": "Management.py executed successfully"}
Response from Management.py: {
  "message": "Management.py executed successfully",
  "status": "success"
}
    
```

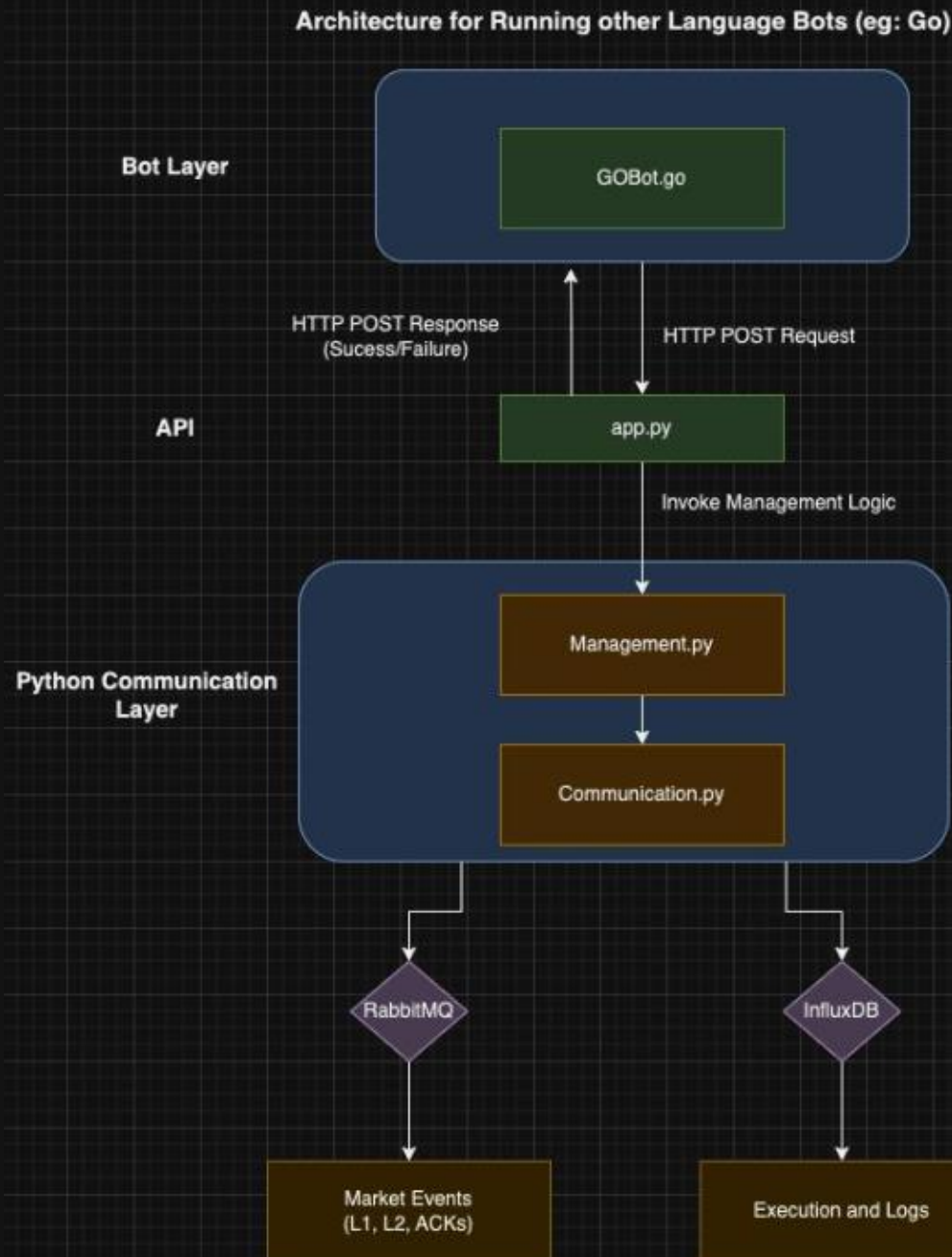
Execution Result





NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES



Other Language Bot Architecture

This architecture supports the integration and execution of bots written in programming languages such as Go. It consists of multiple layers to facilitate seamless communication between the bot, API, and the underlying Python management logic.

This architecture approach leverages the strengths of RabbitMQ for event handling and InfluxDB for logging and analysis, ensuring a robust operational flow.





- **Optimize C++-Python Integration**

Reduce latency in the C++-Python integration to improve overall system performance.

- **Investigate Multithreading and Parallel Processing**

Explore advanced techniques for handling market data and event queues to enhance system scalability.

- **Adapt for Cloud Deployment**

Modify the architecture to enable distributed deployment on cloud platforms for improved scalability and reliability.

- **Containerization and Orchestration**

Leverage Docker and Kubernetes for containerization and orchestration to facilitate efficient scaling and deployment.

- **Web-based Dashboard and Reporting**

Create a real-time monitoring and strategy adjustment dashboard, along with detailed analytics and reporting tools for end-users.

- **Implement Risk Management Features**

Introduce advanced risk assessment modules and automated safeguards against market anomalies to enhance risk management capabilities.

- **Strengthen Security and Compliance**

Enhance security protocols for market data handling and ensure compliance with evolving financial regulations.

- **Improve Documentation and Training**

Enhance user documentation, create video tutorials, and provide comprehensive guides for both developers and end-users.



Conclusion



NYU

COURANT INSTITUTE OF
MATHEMATICAL SCIENCES



Modular Architecture

Implemented a modular design for high flexibility and scalability, enabling future enhancements.



Effective Strategies

Demonstrated successful trading strategies and robust risk management with real-time performance insights.



Strong Foundation

Established a solid foundation for continued development and advancement in algorithmic trading.

The project has delivered a highly adaptable and efficient trading platform with valuable lessons learned. This lays the groundwork for further innovations and expanded capabilities in the future.